

La memoria del sistema. Setup completo.

Memoria persistente + Graphify su un Claude appena installato. Copia, incolla, verifica. Alla fine Claude ricorda tutto e il tuo vault è un grafo interrogabile.

- 01 La mappa — memoria + grafo
- 02 Verifica la memoria automatica
- 03 I 4 tipi di memoria
- 04 Forza la prima memoria
- 05 Installa Graphify — 3 comandi
- 06 Indicizza il vault
- 07 Collega il grafo a Claude (MCP)
- 08 Interroga il grafo — i 3 test
- 09 Cosa indicizzare e cosa no
- 10 Refresh giornaliero automatico
- 11 Checklist finale + troubleshooting

Due pezzi, un cervello

Il problema: ogni chat di Claude parte da zero. La soluzione sono **due sistemi che lavorano insieme** — uno ricorda, l'altro collega.

01 • RICORDA

MEMORY.md

La memoria automatica di Claude Code. File di testo che Claude legge a inizio sessione e aggiorna da solo quando impara qualcosa su di te o sul progetto.

02 • COLLEGA

Graphify

Trasforma il vault in un grafo: ogni concetto è un nodo, ogni relazione un arco. Claude lo interroga via MCP e vede connessioni che tu non vedi.

Regola: MEMORY.md = **elenco** di cose da ricordare. Graphify = **mappa** di come si collegano. Servono entrambi.

02 • MEMORIA AUTOMATICA

Verifica che la memoria sia attiva

La memoria persistente è **già inclusa** in Claude Code — nessuna installazione. Vive in una cartella per ogni progetto. Verifica così, dentro una sessione `claude`:

```
# Dentro Claude Code, chiedi:
> dove sta il tuo file di memoria per questo progetto?

# Risposta attesa: un percorso tipo
# ~/.claude/projects/<nome-progetto>/memory/MEMORY.md
```

Se Claude risponde con il percorso, la memoria è attiva. Il file `MEMORY.md` è l'**indice**: ogni voce punta a un file di dettaglio nella stessa cartella. Si crea da solo alla prima cosa che Claude decide di ricordare.

I 4 tipi di memoria

Claude classifica ogni ricordo in uno di 4 tipi. Conoscerli ti serve per **dettare memorie precise** invece di sperare che capisca da solo.

TIPO • 01

user

Chi sei, che ruolo hai, cosa sai già. Es: "preferisco risposte brevi", "il mio brand vende su Shopify".

TIPO • 02

feedback

Correzioni e conferme sul metodo. Es: "mai sovrascrivere i report", "i tool sempre in inglese".

TIPO • 03

project

Stato del lavoro: deadline, decisioni, contesto. Es: "lo store va live il 15", "il token scade a novembre".

TIPO • 04

reference

Dove stanno le cose fuori dal progetto. Es: "i bug si tracciano su quel foglio", "il gruppo Telegram giusto è questo".

Forza la prima memoria

Non aspettare che succeda da solo: **detta tu la prima memoria** e verifica che persista tra sessioni.

1 Detta la memoria

In sessione: `ricordati che [fatto importante su di te o sul progetto]`. Claude salva il file e aggiorna l'indice MEMORY.md.

2 Chiudi tutto

Esci dalla sessione (`Ctrl+C` o chiudi il terminale). La sessione muore, la memoria no.

3 Riapri e verifica

Nuova sessione `claude` nella stessa cartella → chiedi: `cosa ti ricordi di me?`. Deve citare il fatto del passo 1.

Test passato = memoria operativa. Da ora ogni cosa importante che dici a Claude può sopravvivere alla chat.

Installa Graphify — 3 comandi

Graphify si installa con **uv** (il gestore pacchetti Python veloce). Se non hai uv, il primo blocco lo installa.

```
# 0. Solo se non hai uv (una volta sola)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 1. Installa il tool
uv tool install graphifyy

# 2. Verifica
graphify --help
```

Esce la schermata di help = installato. **Attenzione al nome:** il pacchetto è `graphifyy` (doppia y), il comando è `graphify` (singola).

Indicizza il vault

Punta Graphify alla cartella della conoscenza — **non** a tutto il vault (vedi sezione 09). Per un vault medio ci mette 1–3 minuti.

```
# Indicizza la cartella knowledge del tuo vault
graphify ~/Documents/IlTuoVault/02_KNOWLEDGE

# Output: si crea la cartella graphify-out/ con dentro
#   graph.json   → il grafo (nodi + archi)
#   graph.html   → la visualizzazione interattiva

# Apri il grafo nel browser
open ~/Documents/IlTuoVault/02_KNOWLEDGE/graphify-out/graph.html
```

La prima apertura è il momento "wow": **il tuo cervello visto da fuori**. Click su un nodo = vedi i vicini. Le comunità di nodi = i tuoi temi ricorrenti.

Collega il grafo a Claude

Il grafo nel browser è bello, ma il valore vero è quando **Claude lo interroga da solo**. Si collega via MCP con un comando:

```
# Sostituisci il path con il TUO percorso reale
claude mcp add graphify-vault -- python -m graphify.serve
~/Documents/IlTuoVault/02_KNOWLEDGE/graphify-out/graph.json

# Verifica che il server sia registrato
claude mcp list
```

Da questo momento, in ogni sessione Claude ha i tool del grafo: `query_graph` , `get_neighbors` , `shortest_path` , `god_nodes` . Non devi chiamarli tu — li usa lui quando serve.

Interroga il grafo — i 3 test

Nuova sessione `claude` , tre domande. Se rispondono tutte, il sistema è in piedi.

TEST	CHIEDI	RISPOSTA ATTESA
01 • god nodes	"quali sono i concetti più collegati del mio grafo?"	Lista dei nodi con più connessioni — i tuoi temi centrali
02 • vicini	"cosa è collegato a [un tuo concetto]?"	I nodi adiacenti, con il tipo di relazione
03 • percorso	"come si collegano [concetto A] e [concetto B]?"	Il percorso più corto tra i due, nodo per nodo

Il test 03 è quello che cambia il modo di lavorare: **connessioni tra idee che non sapevi di avere** — su carta non le avresti mai viste.

Cosa indicizzare e cosa no

Indicizzare tutto = grafo rumoroso e inutile. La regola: **dentro la conoscenza, fuori il materiale grezzo.**

DENTRO IL GRAFO	FUORI DAL GRAFO
02_KNOWLEDGE — pattern, decisioni, learnings	04_SESSIONS — log automatici delle sessioni
01_PROJECTS — i progetti attivi e le loro note	03_REFERENCE pesante — PDF, export, dump grezzi
Brains/ — le lezioni dei tuoi esperti	00_INBOX — materiale non ancora processato

Criterio in una frase: se è conoscenza scritta da te o per te → dentro. Se è materiale grezzo o log automatico → fuori.

Refresh giornaliero automatico

Il vault cresce ogni giorno; il grafo deve seguirlo. Niente lavoro manuale: **chiedi a Claude di schedulare il refresh**, è il suo mestiere.

```
# In sessione claude, detta questo:
> schedula un refresh giornaliero del mio grafo Graphify alle 03:00:
  comando "graphify ~/Documents/IlTuoVault/02_KNOWLEDGE --refresh",
  su Mac usa launchd, log su file
```

Claude crea il file di scheduling, lo carica e lo testa. Il `--refresh` è incrementale: riprocessa solo i file cambiati, pochi secondi invece di minuti.

Risultato: ogni mattina il grafo riflette il vault di ieri sera. Tu non tocchi niente.

Checklist finale

Non passare alla Lezione 5 senza tutte e 6 le spunte verdi.

- ☐ **Memoria automatica verificata**
Claude risponde con il percorso del suo MEMORY.md
- ☐ **Prima memoria forzata e persistente**
chiudi sessione → riapri → "cosa ti ricordi di me?" risponde giusto
- ☐ **Graphify installato**
graphify --help risponde
- ☐ **Vault indicizzato**
graphify-out/graph.html si apre e mostra il grafo
- ☐ **MCP collegato**
claude mcp list mostra graphify-vault
- ☐ **I 3 test del grafo passati**
god nodes · vicini · shortest path — tutti rispondono

TROUBLESHOOTING

Se qualcosa si rompe

SINTOMO	CAUSA PROBABILE	FIX
command not found: uv	uv non nel PATH	Chiudi e riapri il terminale; rilancia l'installer uv e segui le righe finali
command not found: graphify	Tool uv non nel PATH	uv tool update-shell , poi riapri il terminale
Indicizzazione lentissima	Cartella troppo grande (PDF, media)	Punta solo a 02_KNOWLEDGE, non a tutto il vault
MCP non appare in claude mcp list	Path del graph.json sbagliato	Verifica che il file esista: ls <path>/graphify-out/graph.json , poi ripeti claude mcp add
Claude non ricorda tra sessioni	Cartella diversa = progetto diverso	La memoria è per-progetto: riapri claude nella stessa cartella
Grafo vuoto o quasi	File non in formato .md	Graphify legge markdown: converti o sposta i .md nella cartella indicizzata

Regola d'oro: qualunque errore — copia il testo **esatto** e incollalo a Claude in chat. Descrivi cosa stavi facendo. Nel 90% dei casi te lo risolve al primo colpo.

Memoria attiva. Ora i cervelli.

Claude ricorda, il vault è un grafo. Nella prossima lezione costruiamo il primo brain interrogabile. · Corso AI + Ecommerce · Lezione 04 → 05